Running Node: Scripts, the REPL, the CLI, and nvm

10 min read · Updated 24 Jul 2026 · Topic 5 of 70

Why it matters

Knowing the concepts is one thing; actually *running* Node fluently is what makes you productive day to day. There is a real difference between a developer who only knows `node app.js` and one who reaches for the REPL to test an idea in two seconds, uses `node -e` in a shell script, keeps the right Node version per project with nvm, and knows the CLI flags that turn a manual dev loop into an automatic one. These are small skills, but together they are the difference between fighting your tools and flowing with them.

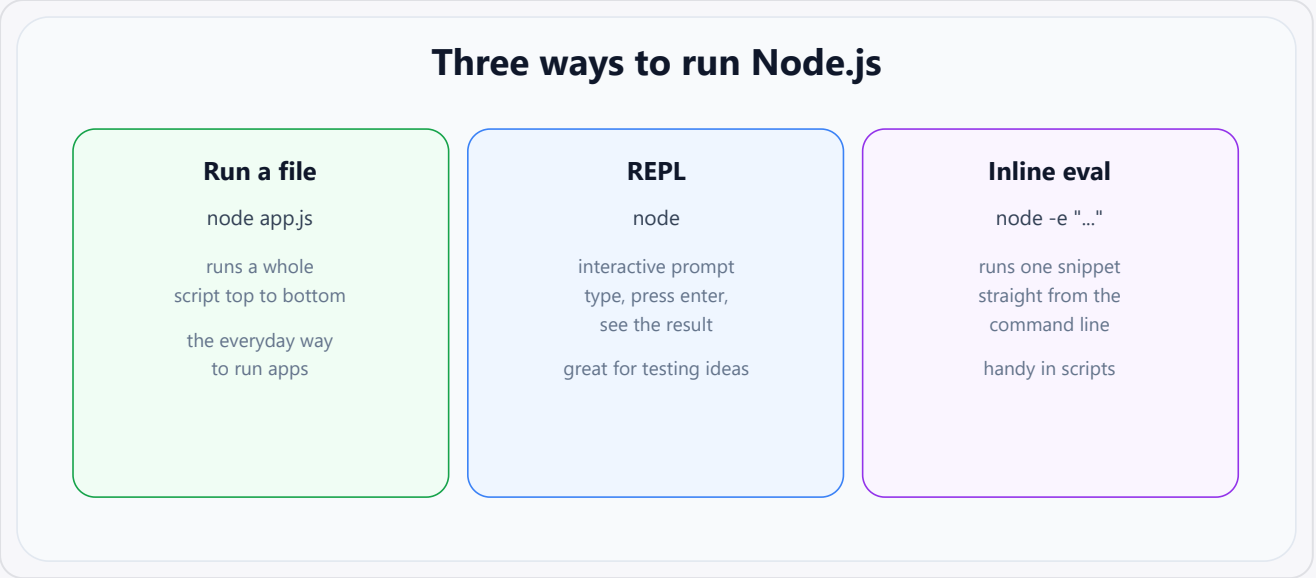
They also come up constantly in interviews and on the job: “How do you check your Node version?”, “How would you quickly test whether this snippet works?”, “How do you run different Node versions for different projects?” This lesson gives you a confident answer to all of them and a mental map of the ways Node executes code.

The concept, simply

There are three everyday ways to run JavaScript with Node, and it helps to picture them as three doors into the same runtime.

1. Run a file: `node app.js` executes a whole script from top to bottom. This is how you run real programs and servers — the everyday default.

2. The REPL: type `node` with no file and you drop into an interactive prompt (Read–Eval–Print Loop). You type an expression, press Enter, and Node immediately evaluates it and prints the result — perfect for experimenting, checking how an API behaves, or doing quick calculations.



Three doors into the runtime: run a file, open the interactive REPL, or evaluate a one-line snippet inline.

On top of *how* you run Node sits *which version* of Node you run. Because projects target different Node versions, a tool called **nvm** (Node Version Manager) lets you install several versions side by side and switch between them per project — so an old client project on Node 18 and a new one on Node 22 can coexist on one machine.

How it works

Running a file. `node path/to/file.js` starts a process, loads the file, runs it, and — importantly — keeps the process alive as long as there is pending asynchronous work (an open server, a timer, a pending Promise). A plain script with no async work exits as soon as the last line runs. This is why a web server "stays running": `server.listen()` keeps the event loop busy. You can pass arguments after the filename (`node app.js --port 8080`), and read them via `process.argv`.

The REPL in depth. Start it with `node`. Each line you type is evaluated and its result printed. Useful REPL features:

- `_` holds the value of the last evaluated expression (like a calculator's answer memory).
- Tab completion works — type `process.` and press Tab to see available properties.

- Multi-line input is supported: an unfinished statement (an open brace or bracket) shows a `...` continuation prompt.
- Dot commands start with a period: `.help`, `.exit` (or Ctrl-D) to quit, `.clear` to reset the context, `.load file.js` and `.save file.js`.
- Core modules are pre-loaded lazily — you can just type `os.platform()` without requiring `os` first.

The REPL is the fastest way to answer "what does this actually return?" without creating a file.

Handy CLI flags. The `node` command has flags worth knowing:

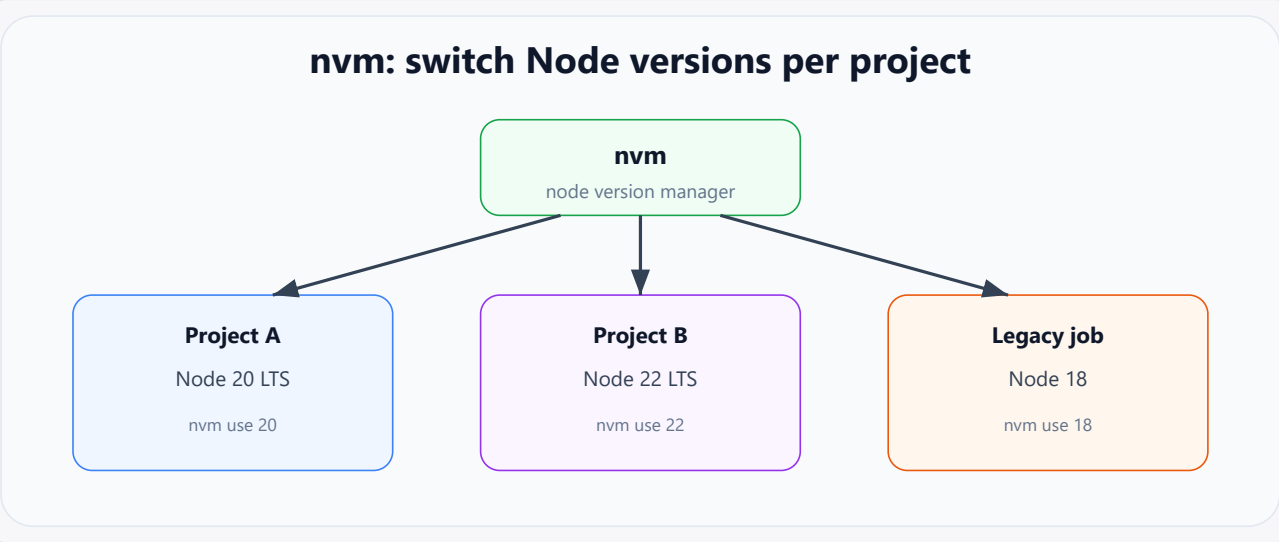
Command / flag	What it does
`node app.js`	Run a script file
`node`	Start the interactive REPL
`node -e "expr"`	Evaluate a snippet inline
`node -p "expr"`	Evaluate *and print* the result
`node --version` / `-v`	Print the Node version
`node --watch app.js`	Re-run automatically when files change (built in, v18.11+)
`node --env-file=.env app.js`	Load env vars from a file (built in, v20.6+)
`node --check app.js`	Syntax-check without running
`node --inspect app.js`	Enable the debugger (Chrome DevTools)

`--watch` and `--env-file` are especially nice: modern Node can auto-restart on save and read a `.env` file without extra packages like `nodemon` or `dotenv` for simple cases.

nvm basics. Node versions matter because language features, performance, and security fixes differ between them, and each project may pin a version. nvm (on

macOS/Linux; nvm-windows or alternatives like `fnm` / `Volta` elsewhere) manages this. The core commands:

Command	Meaning
`nvm install 22`	Install Node 22 (latest of that major)
`nvm install --lts`	Install the latest LTS release
`nvm use 20`	Switch the current shell to Node 20
`nvm ls`	List installed versions
`nvm alias default 22`	Set the default version for new shells



nvm installs multiple Node versions side by side so each project can use the version it was built for.

A common workflow: put an `.nvmrc` file containing a version (e.g. `22`) in a project, and `nvm use` reads it automatically — so everyone on the team runs the same Node. Under the hood, LTS versions are the ones to prefer: even-numbered majors (18, 20, 22) get long-term support and security patches; odd ones are experimental and short-lived.

Code examples

Example 1 — a REPL session. Everything after `>` is what you type.

```
$ node
Welcome to Node.js v22.11.0.
> 2 + 2
4
> _ * 10          // _ is the last result
40
> const os = require('node:os');
undefined
> os.platform()
'darwin'
> [1, 2, 3].map(n => n * n)
[ 1, 4, 9 ]
> .exit           // or press Ctrl-D
```

Example 2 — inline evaluation for quick answers and scripts.

JAVASCRIPT Copy

```
# -e runs code; -p runs AND prints the result:
$ node -p "process.version"
v22.11.0

$ node -p "Math.max(3, 7, 2)"
7

# Useful inside shell scripts / CI:
$ node -e "console.log('Build at', new Date().toISOString())"
```

Example 3 — running a file with the modern dev flags.

JAVASCRIPT Copy

```
// server.js
const http = require('node:http');
const port = Number(process.env.PORT) || 3000;

http.createServer((req, res) => res.end('ok'))
  .listen(port, () => console.log('listening on ' + port));
```

```
// Auto-restart on save + load env from a file — no extra packages:
// node --watch --env-file=.env server.js
// The process stays alive because server.listen keeps the loop busy.
```

Example 4 — checking and switching versions.

JAVASCRIPT Copy

```
# Which Node am I running?
$ node -v
v22.11.0

# Install and pin the LTS for a project:
$ nvm install --lts
$ echo "22" > .nvmrc
$ nvm use
Now using node v22.11.0 (npm v10.x)
```

Common mistakes & gotchas

- **Expecting a plain script to "stay running."** Node exits when there's no pending async work. A script only keeps running if something (a server, an interval) holds the event loop open.
- **Confusing `-e` and `-p`.** `node -e` runs code but prints nothing unless you `console.log`; `node -p` runs it *and* prints the returned value. Reaching for `-e` when you wanted the result is a common slip.
- **Treating REPL results as if they came from your program.** REPL variables and state vanish when you exit — it's a scratchpad, not persistence. Also, a statement like `const x = 5` shows `undefined` (the statement's value), which confuses beginners.
- **Version drift across a team.** Without an `.nvmrc` or pinned version, teammates run different Node versions and hit "works on my machine" bugs. Pin it.
- **Installing global packages under the wrong Node version.** With nvm, global installs are per-version — switch versions and your globally-installed CLI may "disappear" because it lives under the other version.
- **Using odd-numbered Node versions in production.** Odd majors (19, 21, 23) are short-lived and don't get long-term support; stick to LTS (even) majors for anything

- **Reaching for `nodemon` when `--watch` exists.** Modern Node restarts on file change natively; you often don't need an extra dependency for simple cases.

Best practices

- **Use the REPL to learn APIs fast** — when unsure what a method returns, type it into `node` rather than guessing or writing a throwaway file.
- **Pin your Node version per project** with an `.nvmrc` (and an `engines` field in `package.json`) so everyone, including CI, runs the same version.
- **Prefer LTS versions** (even-numbered majors) for production; upgrade deliberately when an LTS line reaches end of life.
- **Lean on built-in flags before adding tools:** `--watch` for auto-restart, `--env-file` for env loading, `--inspect` for debugging — fewer dependencies, less to break.
- **Put run commands in `package.json` scripts** (`npm run dev`) so the exact flags are documented and shared, not re-typed from memory.
- **Know how to exit the REPL** (`.exit` or Ctrl-D, twice-Ctrl-C) so you're never stuck — a small thing that saves real friction.

⭐ Interview lens

Q: "What is the Node REPL and when would you use it?"
A: REPL = Read–Eval–Print Loop, an interactive prompt started by running `node` with no file. You type expressions and see results immediately — ideal for testing snippets, exploring an API, or quick calculations without creating a file. `_` holds the last result. *Trap:* not knowing what the acronym stands for.

Q: "How do you run a one-off JavaScript snippet without a file?"
A: `node -e "code"` to execute it, or `node -p "code"` to execute and print the result. Great for shell scripts and CI checks.

A: A version manager like nvm (or fnm/volta). Install versions with `nvm install`, switch with `nvm use`, and pin per-project with an `.nvmrc` file so the team stays consistent. *Trap:* suggesting you reinstall Node each time — the point is coexistence.

Q: "Why does a Node web server keep running while a plain script exits?"
A: Node exits when the event loop has no more work. A server calls `listen()`, which keeps a handle open and the loop alive; a plain synchronous script finishes its last line with nothing pending, so the process ends.

Q: "What's the difference between LTS and Current Node releases?"
A: Even-numbered majors become LTS (Long-Term Support) with ~30 months of maintenance and security fixes — use these in production. Odd-numbered 'Current' releases are short-lived and carry the newest experimental features. *Trap:* recommending a bleeding-edge odd version for production.

Q: "How would you auto-restart a Node app during development without extra packages?"
A: Use the built-in `node --watch app.js` (Node 18.11+), which restarts on file changes — historically people used `nodemon`, but it's now built in. Pair with `--env-file` to load a `.env` without `dotenv`.

Cheat-card

- **Run a file:** `node app.js` (stays alive while async work is pending).
- **REPL:** `node` → interactive; `_` = last result; `.exit` / Ctrl-D to quit.
- **Inline:** `node -e "code"` runs; `node -p "code"` runs and prints.
- **Version:** `node -v`. **Dev flags:** `--watch`, `--env-file`, `--inspect`, `--check`.
- **nvm:** `install`, `use`, `ls`; pin with `.nvmrc` + `engines`.

- **Versions:** even = LTS (use in prod); odd = short-lived/experimental.

QUICK CHECK — no marks, just for you.

1. What does the `_` variable hold in the Node REPL?

- ◯ The value of the last evaluated expression
- ◯ The path of the current file
- ◯ The most recent error thrown
- ◯ A reference to the global object

Check answer

2. What is the difference between `node -e` and `node -p`?

- ◯ `-e` prints the result automatically; `-p` does not
- ◯ `-e` runs a snippet but prints nothing unless you log; `-p` runs it and prints the returned value
- ◯ They are identical aliases
- ◯ `-p` starts the REPL while `-e` runs a file

Check answer

3. Which tool lets several Node versions coexist so each project can use the version it was built for?

- ◯ npm
- ◯ npx

V8

Check answer

Previous
The Global Object, process, and Environment

Next
The Event Loop Phases

LEARN MORE

ARTICLES & DOCS

- How to use the Node.js REPL
  nodejs.org
- Run Node.js scripts from the command line
  nodejs.org
- REPL | Node.js API Documentation
  nodejs.org

VIDEOS



Node.js Crash Course
Traversy Media

Hand-picked references for this lesson.